

Software Engineering — Lab 3

Component Modelling & Architectural Pattern Selection

Name: Tanmay Sridhar SRN: PES1UG25CS849
Assigned System: Public Bus Live Tracking & Crowding Estimator
Repository: github.com/Tanny23-pixel/SE-Labs--PES1UG25CS849-

1. Requirements & Challenge Analysis

The Public Bus Live Tracking & Crowding Estimator ingests real-time GPS telemetry and passenger-count data to provide commuters and fleet controllers with live bus locations, stop-level arrival estimates (ETA), and crowding levels. Key requirements identified in Lab 1:

- **FR001** — Ingest bus GPS coordinates every 5 seconds and recalculate ETAs for upcoming stops.
- **FR002** — Allow commuters to view the current location of buses on a selected route.
- **FR003** — Calculate and display bus crowding level (Low/Moderate/High/Full) from ticketing sensor data.
- **FR004** — Allow fleet controllers to monitor live location, ETA, and crowding of active buses.
- **FR005** — Allow commuters to search a route and view its upcoming stops and ETAs.
- **NFR001** — Handle telemetry from up to 1,000 active buses simultaneously with average processing latency $\leq$ 2 seconds.

Key challenges:

- **Performance/Scale** — 1,000 concurrent GPS + ticketing-sensor streams must be processed within a strict 2-second average latency budget.
- **Usability** — commuters and fleet controllers need a simple, responsive view of location, ETA, and crowding without exposure to backend complexity.
- **Reliability/Fault isolation** — a fault in one data stream (e.g., a ticketing sensor outage affecting crowding estimation) should not block core location/ETA tracking (FR001, FR002).
- **Data freshness** — ETA and crowding values must always reflect the latest telemetry and never display stale information as current (see UC-02 alternate flow).

2. Architectural Style Comparison

Style	Pros (for this scenario)	Cons (for this scenario)	Suitability
Layered	Clear separation of presentation/business/data; simple to understand and build.	Every request traverses layers, adding overhead; hard to scale ETA computation and GPS ingestion independently at 1,000-bus load.	Low — cannot meet NFR001 scaling needs cleanly.
Microservices	Telemetry ingestion, ETA calculation and crowding estimation scale and fail independently; fits a high-throughput, multi-stream real-time system.	Higher operational complexity and inter-service network latency to manage.	High — directly addresses NFR001 and fault isolation.

Client-Server	Centralized control and simple deployment; easy data consistency.	Single server is a bottleneck/single point of failure under 1,000 simultaneous vehicle streams; limited horizontal scalability.	Low — does not scale to city-wide fleet load.
----------------------	---	---	---

3. Architecture Selection & Justification

We chose the Microservices Architecture for the Public Bus Live Tracking & Crowding Estimator System.

Reason 1 — Independent scaling for real-time load (NFR001):

GPS telemetry ingestion (every 5 seconds, up to 1,000 buses) and ETA/crowding computation have very different and spiky load profiles. As independent services, the Telemetry Ingestion and Crowding Estimation components can each be scaled horizontally on their own to keep the 2-second average processing latency, without over-provisioning the entire system.

Reason 2 — Fault isolation across independent data streams:

GPS data (from the Bus GPS Device/System) and passenger-count data (from the Ticketing Sensor System) arrive from separate external sources. Splitting Telemetry Ingestion, ETA Calculation and Crowding Estimation into separate services means a failure or delay in the ticketing-sensor pipeline degrades only crowding information, while the higher-priority location and ETA features (FR001, FR002) required by commuters and fleet controllers keep working.

Security Advantage:

Each microservice exposes only a narrow, well-defined interface (e.g., the Bus Status Interface) behind a single API Gateway. Raw GPS and ticketing-sensor data never reach the commuter or fleet-controller clients directly, the Data Store component is not exposed externally at all, and each service boundary can enforce its own authentication/authorization, reducing the overall attack surface compared to a monolithic, single-server design.

Performance Benefit:

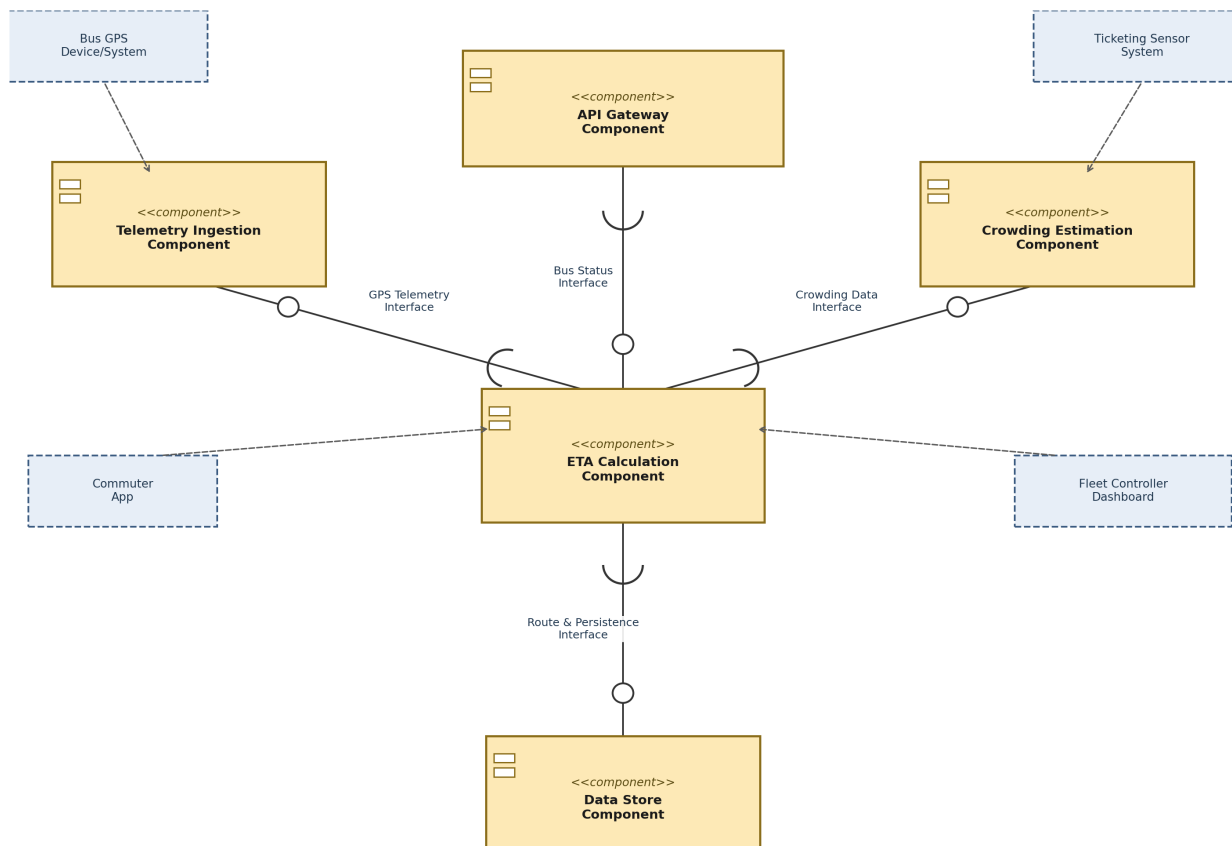
Because the Telemetry Ingestion, ETA Calculation and Crowding Estimation components are independently deployable, the busiest component under peak load (telemetry ingestion at 1,000 concurrent buses) can be scaled out without redeploying or slowing down the rest of the system, directly supporting the NFR001 requirement of ≤ 2 -second average processing latency even at full fleet capacity.

4. Component Identification (5 Components)

Component	Responsibility
Telemetry Ingestion Component	Receives and validates raw GPS coordinates from the Bus GPS Device/System every 5 seconds (FR001) and forwards clean location data.
ETA Calculation Component	Core service that computes stop-level arrival estimates from telemetry, aggregates location, ETA and crowding into a single bus-status view, and serves route/stop searches (FR001, FR002, FR005).
Crowding Estimation Component	Processes passenger-count data from the Ticketing Sensor System and computes a Low/Moderate/High/Full crowding level per bus (FR003).
Data Store Component	Persists route/stop definitions, schedules and telemetry history used for ETA calculation and route search.
API Gateway Component	Exposes bus status, route-search and fleet-monitoring endpoints to the Commuter App and Fleet Controller Dashboard (FR002, FR004, FR005).

5. UML Component Diagram

ETA Calculation Component acts as the central hub, mirroring the reference layered example: it requires GPS data, crowding data and persisted route data, and provides an aggregated Bus Status Interface consumed by the API Gateway.



Public Bus Live Tracking & Crowding Estimator — UML Component Diagram (Microservices Architecture)

Required Interfaces (exactly 4)

#	Interface	Provider (ball) → Consumer (socket)	Purpose
1	GPS Telemetry Interface	Telemetry Ingestion → ETA Calculation	Supplies latest bus location/speed for ETA computation.
2	Crowding Data Interface	Crowding Estimation → ETA Calculation	Supplies current crowding level for aggregation into bus status.
3	Route & Persistence Interface	Data Store → ETA Calculation	Provides route/stop data and stores telemetry history.
4	Bus Status Interface	ETA Calculation → API Gateway	Exposes aggregated location, ETA and crowding to client apps.

Submitted via GitHub repository: github.com/Tanny23-pixel/SE-Labs--PES1UIG25CS849- (Lab 3 folder).